

The Request → Response Cycle

What actually happens between hitting Enter and seeing pixels — mapped to your React / Next.js stack.

Why this matters: Every `fetch()`, every form submission, every page navigation triggers this cycle. Knowing each step turns a 3-hour debugging session into a 30-second diagnosis.

1. The 8 stages, end to end

1 URL parsing
protocol / domain / port / path / query / fragment — the fragment never reaches the server

2 DNS lookup
Converts the domain name to an IP address. Cached at browser, OS, router, ISP.

3 TCP handshake
Three-way handshake (SYN, SYN-ACK, ACK) opens a reliable connection.

4 TLS handshake (HTTPS)
Server proves identity via certificate, both sides derive a shared key, everything afterward is encrypted.

5 HTTP request sent
Request line (method + path), headers (auth, content-type, cookies), and optional body.

6 Server processes
Reverse proxy routes → middleware (auth, CORS, rate limit) → route handler → database query.

7 Server responds
Status code + headers + body. Status tells you how it went, headers carry metadata, body carries data.

8 Browser renders
Parse HTML → build DOM → fire sub-requests for assets → paint → run JavaScript.

2. Anatomy of a URL

https://api.example.com:443/users/42?ref=email#bio
| | | | |
protocol domain port path query fragment

Part	Example	Purpose
Protocol	https	How to talk (HTTPS, WS, FTP)
Domain	api.example.com	Which server (resolved via DNS)
Port	443	Which door (80 = HTTP, 443 = HTTPS)
Path	/users/42	Which resource
Query	?ref=email	Extra parameters (filters, search)
Fragment	#bio	Browser-only — never sent to server

3. HTTP methods

Method	Meaning	Body?	Idempotent ?	Example
GET	Fetch data	No	Yes	GET /api/users/42
POST	Create / submit	Yes	No	POST /api/users
PUT	Replace entire resource	Yes	Yes	PUT /api/users/42
PATCH	Update fields	Yes	Usually no	PATCH /api/users/42
DELETE	Delete resource	Optional	Yes	DELETE /api/users/42

Idempotent means calling it 5 times has the same effect as calling it once. GET / PUT / DELETE should be idempotent. POST is not — 5 POSTs = 5 new records.

4. Status codes you must know cold

Code	Meaning	When you see it
200 OK	Success	Standard happy path
201 Created	Resource created	After POST that made something
204 No Content	Success, no body	DELETE that worked
301 Moved Permanently	Permanent redirect	URL changed forever
302 Found	Temporary redirect	Login flow redirects
304 Not Modified	Use your cache	Browser caching
400 Bad Request	Client sent garbage	Validation failed
401 Unauthorized	Not authenticated	No / bad credentials

Code	Meaning	When you see it
403 Forbidden	Authed, not allowed	Lacks permission
404 Not Found	Resource missing	Wrong URL or ID
409 Conflict	State conflict	Duplicate email signup
422 Unprocessable Entity	Semantic validation failed	Form validation in REST APIs
429 Too Many Requests	Rate limited	Too many calls
500 Internal Server Error	Server crashed	Unhandled exception
502 Bad Gateway	Upstream broke	Reverse proxy issue
503 Service Unavailable	Server overwhelmed	Downtime / overload
504 Gateway Timeout	Upstream too slow	DB or API timed out

The mental rule:

2xx = success 3xx = redirect 4xx = *you* (client) screwed up 5xx = *server* screwed up

5. Common headers you'll actually use

Header	Direction	Purpose
Content-Type	Both	Body format (application/json, multipart/form-data, text/event-stream...). See section 6.
Authorization	Request	Auth token (Bearer eyJ...)
Cookie / Set-Cookie	Req / Resp	Session, preferences
Cache-Control	Both	Cache behavior (no-store, max-age=3600)
Accept	Request	What formats the client wants
Origin	Request	Where the request is coming from (CORS)
Access-Control-Allow-Origin	Response	CORS: which origins are allowed
Content-Length	Both	Size of body in bytes
User-Agent	Request	Browser / client identifier
Location	Response	Where to redirect to (with 3xx status)

6. Content-Type values you'll actually see

Content-Type **tells the receiver how to parse the body**. Pick the right one and the receiver knows whether to JSON . parse, decode form fields, stream tokens, or save bytes as a file.

Content-Type	What it carries	Where you meet it
application/json	JSON body	REST APIs, fetch() with JSON body — your daily default
application/x-www-form-urlencoded	Form fields, URL-encoded	Classic HTML <form> submits, OAuth token endpoints
multipart/form-data	Files + fields, boundary-separated	File uploads, Next.js Server Actions
text/plain	Plain text	Simple uploads, debugging
text/html	HTML page	Browser navigations, SSR responses
text/css	Stylesheet	<link rel="stylesheet">
text/javascript / application/javascript	JS file	<script src="...">
text/event-stream	Server-Sent Events (SSE) stream	Streaming chat tokens, live notifications, AI integrations
application/octet-stream	Raw binary	File downloads of unknown type
application/pdf, image/png, image/jpeg, ...	Specific file types	Static assets, generated files

SSE primer (text/event-stream)

Server keeps a connection open and pushes a stream of text events to the client over a single HTTP response. **One-way: server → client.** Heavily used in AI streaming (ChatGPT, Claude, Vercel AI SDK) to push tokens as they're generated.

Server (Next.js route handler)

```
export async function GET() {
  const stream = new ReadableStream({
    async start(controller) {
      for (const chunk of ['Hello', ' world', '!']) {
        controller.enqueue(`data: ${chunk}\n\n`); // SSE format
        await new Promise(r => setTimeout(r, 500));
      }
      controller.close();
    },
  });
  return new Response(stream, {
    headers: { 'Content-Type': 'text/event-stream' }, // ← the key
  });
}
```

Client

```
const es = new EventSource('/api/stream');
es.onmessage = (e) => console.log(e.data); // 'Hello', then ' world', then '!'
```

SSE vs alternatives

	SSE	WebSocket	Polling
Direction	Server → Client only	Both ways	Client → Server (repeated)
Protocol	Plain HTTP	Upgraded ws://	Plain HTTP
Auto-reconnect	Yes (built-in)	No (manual)	N/A
Use for	Streaming AI tokens, notifications	Chat (both ways), games	Simple "any updates?" checks
Complexity	Low	High	Lowest

For AI integration work in Next.js, SSE is the right choice 90% of the time. WebSockets are overkill unless the client also needs to push live data back.

7. What a real request and response look like

Request

```
POST /api/users HTTP/1.1
Host: api.example.com
Content-Type: application/json
Authorization: Bearer eyJhbGciOi...
Cookie: session=abc123

{ "name": "Akshay", "email": "akshay@example.com" }
```

Response

```
HTTP/1.1 201 Created
Content-Type: application/json
Set-Cookie: session=xyz; HttpOnly; Secure
Cache-Control: no-store

{ "id": 42, "name": "Akshay" }
```

8. CORS in two sentences

Browsers block JavaScript from *reading* responses from a different origin than the page it's running on, unless that server explicitly allows it via the Access-Control-Allow-Origin header.

CORS is a **browser policy**, not a network thing. The server actually responds, but the browser hides the response from your JS code if the headers don't welcome your origin.

9. Cookies vs localStorage for auth

Property	Cookie	localStorage
Sent automatically with every request	Yes	No
Readable by JavaScript	Yes (unless HttpOnly)	Yes
Vulnerable to XSS	If readable, yes	Yes
Vulnerable to CSRF	Yes (need protections)	No
Cross-tab	Yes	Yes
Survives reload	Yes	Yes
Best for	Sessions, auth tokens (HttpOnly)	Theme prefs, non-sensitive UI state

Modern best practice: store JWTs in HttpOnly cookies (set by the server), not localStorage. localStorage tokens are stealable by any XSS attack.

10. How it maps to your Next.js stack

```
Browser (Client Component) ----- 5. HTTP request
                                   |
Vercel Edge / Server               |
|-- middleware.ts                  6. Middleware (auth, redirects)
|-- route handler                  6. Business logic
|-- Prisma query                   6. DB query
|-- NextResponse.json()           7. Build response

Browser receives
|-- React renders                  8. Paint
|-- Hydration
```

Security reminder: never trust client-provided IDs for "who am I?" queries — always read the user ID from the server session, not from ?userId=. Otherwise anyone can ask for anyone else's data.

11. The 15-line preflight + authentication example

This is the cycle in real code — a client on one origin, an API on another, with a JWT and a preflight handshake in front of the real request.

Client — runs on `http://localhost:3000`

```
await fetch('https://api.example.com/users/42', {
  method: 'PATCH', // non-simple → triggers preflight
  headers: {
    'Content-Type': 'application/json', // custom header → triggers preflight
    'Authorization': 'Bearer eyJhbGciOi...', // auth token
  },
  credentials: 'include', // also send cookies
  body: JSON.stringify({ name: 'Akshay' }),
});
```

Server — Next.js route handler

```
export async function PATCH(req) {
  const token = req.headers.get('authorization')?.split(' ')[1];
  const user = verifyJWT(token); // 401 if invalid
  if (!user) return new Response('Unauthorized', { status: 401 });
  const body = await req.json();
  return Response.json({ id: 42, ...body });
}
```

What actually goes over the wire

1. Browser: OPTIONS /users/42 browser)	← PREFLIGHT (auto-fired by browser)
Origin: http://localhost:3000 Access-Control-Request-Method: PATCH Access-Control-Request-Headers: content-type, authorization	
2. Server: 200 OK may"	← server says "yes, you may"
Access-Control-Allow-Origin: http://localhost:3000 Access-Control-Allow-Methods: PATCH Access-Control-Allow-Headers: content-type, authorization Access-Control-Allow-Credentials: true	
3. Browser: PATCH /users/42 Authorization: Bearer eyJ... Content-Type: application/json { "name": "Akshay" }	← REAL REQUEST (now allowed)
4. Server: 200 OK { "id": 42, "name": "Akshay" }	← REAL RESPONSE

One `fetch()` in your code = two round trips on the network. The browser silently inserts the `OPTIONS` preflight whenever the request is "non-simple" — i.e. uses `PATCH/PUT/DELETE`, sends `JSON`, or includes custom headers like `Authorization`.

Common bug: people see the `OPTIONS` request in DevTools and wonder why their backend logs show two requests for one click. That's the preflight — it's not a bug.

12. Do you need to set preflight?

You do **NOT** write preflight code on the client. The browser does it automatically.
But on the **server**, you do have to handle it — otherwise the browser blocks your real request.

The split

Side	What you do
Client (React / Next.js fetch)	Nothing. Just call <code>fetch()</code> normally. Browser fires OPTIONS automatically when needed.
Server (your API)	Must respond to the OPTIONS request with the right CORS headers. Otherwise the browser blocks the real request.

When does the browser send a preflight?

The browser fires a preflight OPTIONS request only if the request is "non-simple". Triggers:

- Method is anything other than GET, HEAD, or POST
- Custom headers like Authorization, X-Custom-Token
- Content-Type is application/json (anything outside form-encoded / plain text / multipart)

• Plain `fetch('/api/users')` with no auth header → **no preflight** (simple request)
• `fetch('/api/users', { headers: { Authorization: '...' } })` → **preflight fires**

Same origin vs cross origin

If client and server are on the **same origin** (same Next.js app), CORS doesn't apply at all — no preflight, no headers needed. This is the common case for Next.js App Router with route handlers.

If they're on **different origins** (separate API server, microservice), your server must handle CORS.

Server-side handler (different origins)

```
// Next.js route handler
export async function OPTIONS() {
  return new Response(null, {
    status: 204,
    headers: {
      'Access-Control-Allow-Origin': 'http://localhost:3000',
      'Access-Control-Allow-Methods': 'GET, POST, PATCH, DELETE',
      'Access-Control-Allow-Headers': 'Content-Type, Authorization',
      'Access-Control-Allow-Credentials': 'true',
    },
  });
}

export async function PATCH(req) {
  // your real logic, but the response also needs the Allow-Origin header
  return Response.json({ ok: true }, {
    headers: { 'Access-Control-Allow-Origin': 'http://localhost:3000' },
  });
}
```

In Express, the `cors` middleware does this for you in one line. In Next.js, put it in `middleware.ts` so you don't repeat the headers in every route.

Mental model

Preflight is the browser asking the server "are you okay if I send this?" You never write the question — the browser does. You only write the answer — and only if your client and API live on different origins.

13. Interview Q&A

Q: Walk me through what happens when I type a URL and hit Enter.

A: URL parse → DNS lookup → TCP handshake → TLS handshake (if HTTPS) → HTTP request sent → server processes (middleware, DB) → response built → browser parses HTML → fires sub-requests for assets → renders → runs JS.

Q: Difference between PUT and PATCH?

A: PUT replaces an entire resource (you send the full object). PATCH updates fields partially (you send only what is changing).

Q: What is an idempotent method?

A: One that produces the same effect no matter how many times you call it. GET, PUT, DELETE are idempotent. POST is not.

Q: Difference between 401 and 403?

A: 401 = you are not authenticated (no / bad credentials). 403 = you are authenticated but lack permission.

Q: What is CORS?

A: A browser security policy that blocks JS from reading cross-origin responses unless the server allows it via Access-Control-Allow-Origin. Non-simple requests trigger a preflight OPTIONS request.

Q: What is text/event-stream used for?

A: Server-Sent Events (SSE). The server keeps the connection open and streams events to the client over plain HTTP. One-way (server → client). Used for streaming AI tokens, live notifications, build logs. The Vercel AI SDK and ChatGPT-style apps use SSE under the hood.

Q: SSE vs WebSocket?

A: SSE is one-way (server → client) over plain HTTP with built-in auto-reconnect. WebSocket is two-way over a separate ws:// protocol with manual reconnect. Pick SSE for streaming pushes; WebSocket only if the client also needs to push live data back.

Q: Do I need to write preflight code on the client?

A: No. The browser sends the OPTIONS preflight automatically for non-simple requests. You only need to handle it on the server (when client and API are on different origins).

Q: What triggers a preflight?

A: Method other than GET/HEAD/POST, custom headers like Authorization, or Content-Type of application/json. A plain GET with no custom headers does not trigger one.

Q: Why use HTTPS over HTTP?

A: Encrypts data in transit, prevents eavesdropping and tampering, and proves server identity via certificates.

Q: Cookie vs localStorage for storing tokens?

A: Cookies are sent automatically and can be HttpOnly (safer from XSS) but vulnerable to CSRF. localStorage is JS-accessible (XSS vulnerable) but not auto-sent (CSRF-safe). Best: HttpOnly cookies + CSRF protection.

Q: What does keep-alive do?

A: Reuses the same TCP/TLS connection for multiple requests instead of reopening for each.

Q: HTTP/2 vs HTTP/1.1?

A: HTTP/2 multiplexes many requests over one TCP connection, uses header compression, and supports server push.

One-line mental model

A request is a structured letter you send to a specific computer asking for a specific resource. A response is the structured reply, with a status code that says how it went. Everything else — DNS, TCP, TLS, headers, middleware, preflight, DB queries — is plumbing around that letter exchange.